



WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET	V2.0.6	DOWNLOADS	1.1K	LICENSE	MIT	GITHUB PAGES	ONLINE
-------	--------	-----------	------	---------	-----	--------------	--------

◆ Gestione centralizzata dei report RDLC

1. Scopo generale del codice

Il file **ReportManager.vb** contiene una classe *singleton* (`NotInheritable`) che centralizza tutte le operazioni relative ai report RDLC in un'applicazione Windows Forms.

Le principali funzionalità sono:

- **Aprire** un report RDLC in un `ReportViewer` a partire da una `DataTable` o da una query SQL.
- **Aggiornare** o **resettare** il `ReportViewer` .
- **Esportare** il report corrente in formato PDF, con possibilità di aprirlo automaticamente.
- **Avviare** la finestra di stampa del `ReportViewer` .

In questo modo il resto dell'applicazione può richiamare metodi statici senza preoccuparsi della configurazione interna del viewer o della gestione dei file.

2. Componenti chiave e funzioni

Sezione	Metodo	Descrizione
Apri Report	<code>ApriReport(viewer, reportName, table, dataSourceName)</code>	Carica un report RDLC in un <code>ReportViewer</code> usando una <code>DataTable</code> .
	<code>ApriReport(viewer, reportName, query, dataSourceName, parameters)</code>	Esegue una query SQL, riempie una <code>DataTable</code> e chiama il metodo precedente.
Gestione Viewer	<code>Refresh(viewer)</code>	Richiama <code>RefreshReport()</code> sul viewer.
	<code>Clear(viewer)</code>	Resetta il viewer (<code>Reset()</code>).

Sezione	Metodo	Descrizione
Stampa PDF	<code>EsportaPdf(reportName, table, dataSourceName, apriPdf)</code>	Esporta il report in PDF, salva su disco e opzionalmente lo apre.
	<code>Stampa(viewer)</code>	Mostra la finestra di stampa del viewer.

Dettagli di implementazione

- **CartellaReport** – costante che indica la sottocartella “Reports” dove sono conservati i file `.rdlc`.
- **Path.Combine** – costruisce il percorso completo del report.
- **File.Exists** – verifica l’esistenza del file prima di procedere.
- **ReportViewer** – viene resettato, impostato su `ProcessingMode.Local`, e configurato con il percorso del report e la fonte dati (`ReportDataSource`).
- **Zoom e DisplayMode** – impostati su `PrintLayout` e `PageWidth` per una visualizzazione ottimale.
- **Esportazione PDF** – utilizza `LocalReport.Render("PDF")` per ottenere un array di byte, che viene poi scritto su disco con `File.WriteAllBytes`.
- **Process.Start** – apre il PDF appena creato se `apriPdf` è `True`.
- **LogLeggiScrivi.ScriviLogMsg** – chiamata a un logger interno per registrare errori e operazioni.

3. Flusso di esecuzione

3.1 Apertura di un report con `DataTable`

1. **Validazione:** controlla che `viewer`, `table`, `reportName` e `dataSourceName` non siano nulli o vuoti.
2. **Percorso:** costruisce il percorso completo del file RDLC.
3. **Verifica file:** lancia `FileNotFoundException` se il file non esiste.
4. **Configurazione viewer:**
 - `Reset()` → `ProcessingMode.Local` → `LocalReport.ReportPath` → `DataSources.Clear()` → `DataSources.Add(...)`.
5. **Visualizzazione:** imposta `DisplayMode.PrintLayout`, `ZoomMode.PageWidth` e chiama `RefreshReport()`.
6. **Gestione eccezioni:** registra l’errore con `LogLeggiScrivi`.

3.2 Apertura di un report con query SQL

1. **Esecuzione query:** `DB.FillDataTable(query, parameters)` restituisce una `DataTable`.

2. **Chiamata:** passa la `DataTable` al metodo di apertura precedente.
3. **Gestione eccezioni:** registra l'errore e rilancia un `Exception` con messaggio più chiaro.

3.3 Esportazione PDF

1. **Validazione:** simile al metodo di apertura.
2. **Percorso:** costruisce il percorso del report.
3. **SaveFileDialog:** chiede all'utente dove salvare il PDF.
4. **Creazione `LocalReport`:** imposta `ReportPath`, aggiunge la fonte dati e renderizza in PDF.
5. **Scrittura su disco:** `File.WriteAllBytes`.
6. **Log:** registra il percorso del file creato.
7. **Apertura PDF:** se `apriPdf` è `True`, avvia il lettore PDF predefinito.
8. **Eccezioni:** log degli errori.

3.4 Stampa

1. **Verifica:** se il viewer è `Nothing`, esce.
2. **PrintDialog:** mostra la finestra di stampa.
3. **Log:** registra l'avvio della stampa.
4. **Eccezioni:** log degli errori.

◆ Conclusioni

Il `ReportManager` è un helper centralizzato che semplifica la gestione dei report RDLC in un'applicazione Windows Forms.

Grazie ai metodi statici, il resto del codice può aprire, stampare o esportare report con poche righe di chiamata, lasciando a `ReportManager` la responsabilità di configurare il viewer, gestire i file e registrare gli errori.

Con alcune piccole ottimizzazioni (gestione delle eccezioni, configurabilità, thread-safety) il componente può diventare ancora più robusto e flessibile.